

Classification: Decision Trees

Problem Setup, Python Implementation, Decision Boundary, and Model Evaluation

Xu Shirong

School of Economics, Xiamen University

May 12, 2026

Lecture Roadmap

- 1 Classification Problem Setup
- 2 Decision Trees
- 3 Learning a Tree
- 4 Python Implementation Options
- 5 Applied Case: Loan Default
- 6 Evaluation
- 7 Wrap-Up

Machine Learning: General Framework

- 1 Start with data collected from the real problem.
- 2 Split data into training data and testing data.
- 3 Learn a model from the training data.
- 4 Use the learned model to predict labels for testing data.
- 5 Evaluate whether the model generalizes.

Core workflow

Training data → Learning algorithm
→ Model → Test predictions

Main question

Can the model perform well on data it did not see during training?

Training Data vs. Testing Data

Data Part	Role
Training data	Used by the algorithm to learn model parameters, split rules, or decision patterns.
Testing data	Held out until the end and used to estimate performance on unseen observations.

Why independence matters

If the testing data influences training, the evaluation becomes too optimistic because the model has already seen information from the test set.

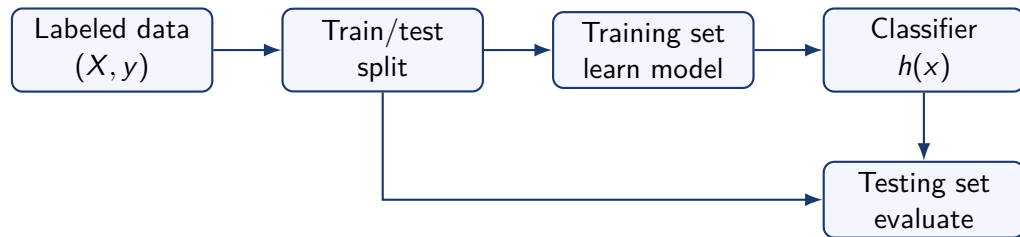
What Can Go Wrong Without a Clean Test Set?

- The model may memorize training examples instead of learning a reusable pattern.
- Test accuracy may look high because test information leaked into training.
- Hyperparameters chosen directly on the test set make the test set part of the training process.
- A model that looks good in slides or notebooks may fail on future data.

Rule of thumb

Train on training data. Choose final performance claims from independent testing data.

Batch Supervised Learning Pipeline



- The same pipeline applies to decision trees, logistic regression, random forests, and many other classifiers.
- In this lecture, the classifier $h(x)$ is a decision tree.

Why Classification Matters

- Many prediction tasks ask for a category rather than a number.
- Examples: loan default or no default, spam or not spam, disease or no disease, churn or stay.
- A classifier learns from labeled examples and predicts labels for future observations.
- Decision trees are a good starting point because the model is a sequence of readable rules.

Core message

Classification is supervised learning where the output space is discrete.

Supervised Learning Setup

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n, \quad x^{(i)} \in X, \quad y^{(i)} \in Y.$$

- $x^{(i)}$: feature vector, such as income, credit score, or exam score.
- $y^{(i)}$: observed label, such as default/no default.
- $f : X \rightarrow Y$: unknown target function.
- $h \in \mathcal{H}$: learned hypothesis that approximates f .

Learning goal

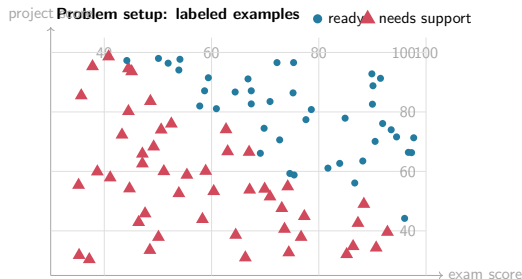
Use labeled training data to build a classifier that performs well on new, unseen data.

Classification Workflow Checklist

- ① Define the prediction target and label coding.
- ② Choose features and check their meaning.
- ③ Split data into training and test sets.
- ④ Train a model on the training data.
- ⑤ Visualize the learned rule or decision boundary when possible.
- ⑥ Evaluate the model with a confusion matrix and metrics.

Running Example: Student Readiness

- Goal: classify whether a student is ready.
- Features:
 - exam_score
 - project_score
- Label:
 - ready
 - needs_support
- Data file:
output/student_readiness_dataset.csv



What Is a Decision Tree?

- Each internal node tests one feature.
- Each branch corresponds to an answer to the test.
- Each leaf gives a predicted class label.
- A prediction is made by following one path from the root to a leaf.

Example rule

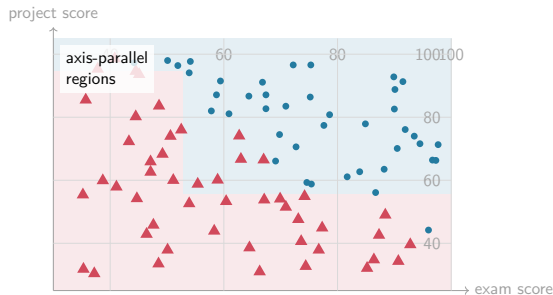
If `project_score > 55.5` and `exam_score > 53.2`, predict ready.

Continuous Features Need Thresholds

- For a continuous feature, a node usually checks whether $x_j \leq t$.
- Example: `credit_score <= 620`.
- The algorithm searches candidate thresholds and chooses the one that best separates the classes.
- This gives decision trees a simple but flexible rule structure.

Decision Boundary

- A split on feature 1 creates a vertical line.
- A split on feature 2 creates a horizontal line.
- Multiple splits form axis-parallel rectangles.
- Each region receives the label of one leaf node.



Expressiveness and Tree Depth

- A shallow tree is easy to explain but may underfit.
- A deep tree can fit complex patterns but may overfit.
- Increasing depth increases the size of the hypothesis space.
- Practical implementations control depth, leaf size, and impurity reduction.

Important tradeoff

Interpretability and flexibility often move in opposite directions.

Top-Down Tree Learning

- ① Put all training observations at the root.
- ② Evaluate possible feature-threshold splits.
- ③ Choose the split that most improves node purity.
- ④ Send observations to child nodes.
- ⑤ Repeat recursively until a stopping rule is reached.

Stopping rules

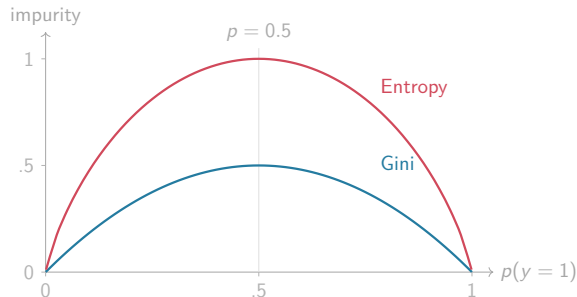
Maximum depth, minimum leaf size, or no useful impurity reduction.

Impurity: What Makes a Node Good?

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

$$\text{Entropy}(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

- Pure node: all labels are the same.
- Mixed node: labels are spread across classes.
- Lower impurity means a cleaner classification rule.



For a candidate split of node S into S_L and S_R :

$$\text{Gain} = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R).$$

- $I(\cdot)$ can be entropy or Gini impurity.
- A good split has high parent impurity and low child impurity.
- The algorithm greedily chooses the best split at the current node.

Python Step 1: Build Data and Split

```
X, y = make_dataset(seed=7)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, seed=13
)
```

- Training set is used to learn split rules.
- Test set is kept unseen until evaluation.
- The script writes the dataset and all figures used in this deck.

Python Step 2: Compute Impurity

```
def gini(y):  
    p = np.bincount(y, minlength=2) / len(y)  
    return 1.0 - np.sum(p * p)  
  
def entropy(y):  
    p = np.bincount(y, minlength=2) / len(y)  
    return -sum(pi * math.log2(pi) for pi in p if pi > 0)
```

Interpretation

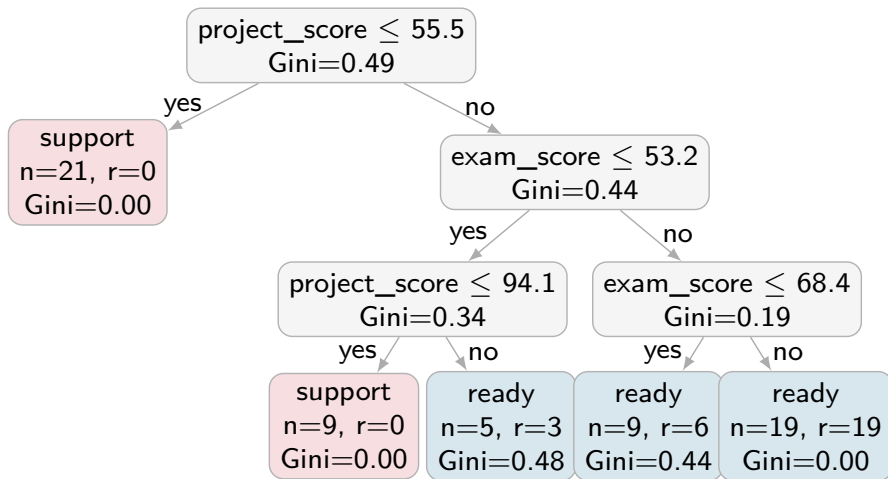
Both functions are small when the node is pure and large when the node is highly mixed.

Python Step 3: Search Candidate Thresholds

```
for feature in range(X.shape[1]):
    values = np.unique(X[:, feature])
    thresholds = (values[:-1] + values[1:]) / 2
    for threshold in thresholds:
        left = X[:, feature] <= threshold
        weighted = (
            left.sum() * gini(y[left])
            + (~left).sum() * gini(y[~left])
        ) / len(y)
        gain = gini(y) - weighted
```

- Every candidate split is evaluated by the same formula.
- The best split is stored in the tree node.

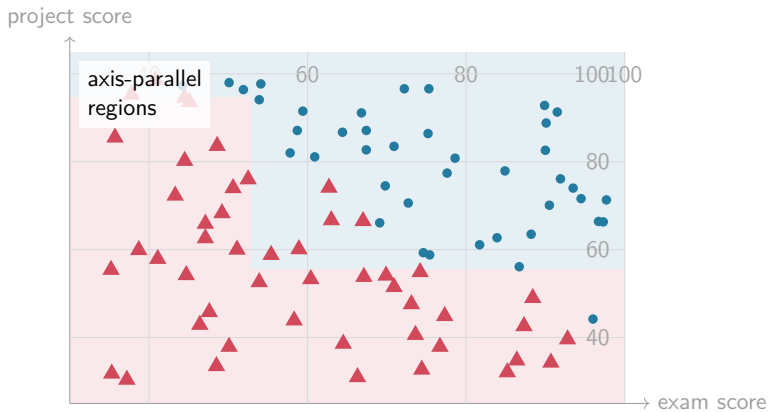
Learned Tree: Student Example



Reading the Learned Tree

- The root split is `project_score <= 55.5`.
- Low project score is immediately classified as `needs_support`.
- Other observations move to deeper rules using exam score and project score.
- Each leaf reports sample count, positive count, and impurity.

Student Decision Boundary



- Blue regions predict ready.
- Orange regions predict needs_support.

Option A: Teaching Implementation

```
tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=5)
tree.fit(X_train, y_train)
y_pred = tree.predict(X_test)
```

- This deck uses a small hand-written decision tree implementation.
- Advantage: every step is visible for teaching.
- Limitation: it only supports simple numeric binary classification.

Option B: scikit-learn Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report

clf = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_leaf=5,
    random_state=13
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

When to use this

Use scikit-learn for real projects; use the hand-written version to understand the mechanics.

Loan Default Prediction

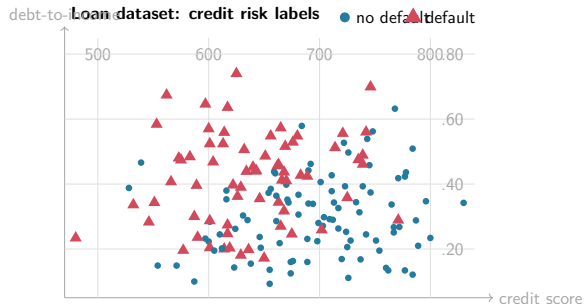
- Business question: can we identify borrowers with higher default risk?
- Label: `default` vs `no_default`.
- Features in this classroom dataset:
 - `credit_score`
 - `debt_to_income`
- Data file: `output/loan_default_dataset.csv`

Interpretation warning

This is a classroom loan-risk dataset generated for demonstration. A real credit model needs stronger validation, bias checks, and regulatory review.

Loan Dataset: First Look

- Lower credit score generally increases risk.
- Higher debt-to-income ratio generally increases risk.
- The classes overlap, so no single perfect rule exists.



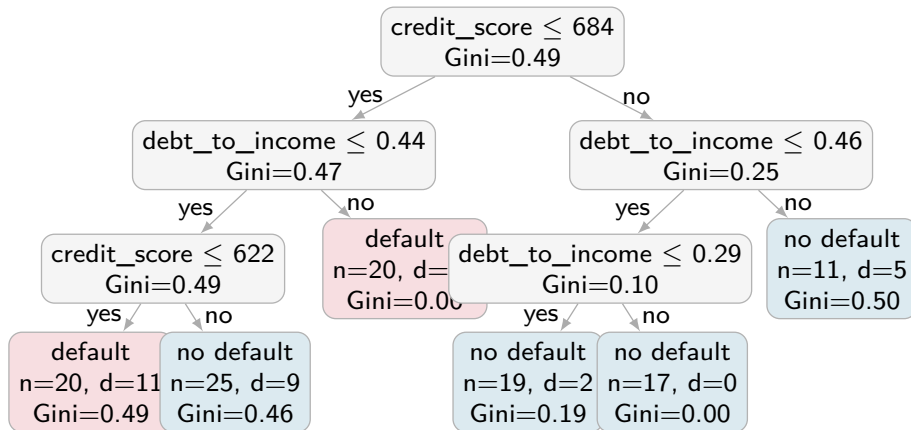
Loan Case: Code Structure

```
X, y = make_loan_dataset()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, seed=31
)

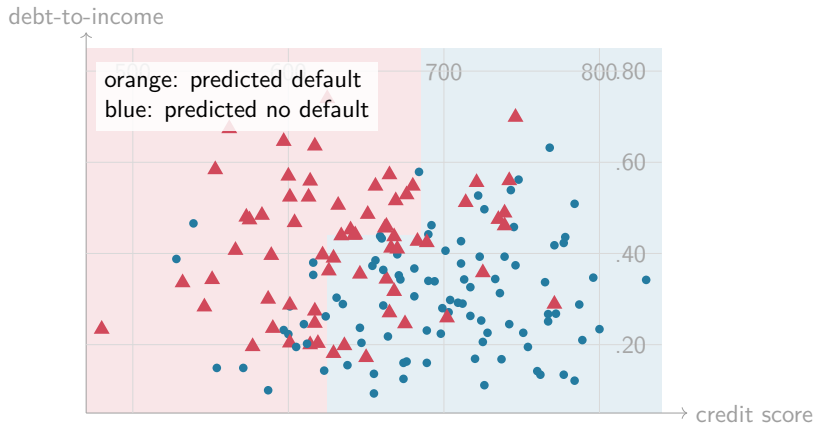
tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=8)
tree.fit(X_train, y_train)
y_pred = tree.predict(X_test)
```

- Same workflow, different dataset.
- This is the key habit: keep modeling steps consistent.

Loan Case: Learned Tree



Loan Case: Decision Boundary



- Orange regions predict default.
- Blue regions predict no_default.

Loan Case: What the Boundary Tells Us

- The model combines credit score and debt-to-income ratio through threshold rules.
- High debt-to-income areas become risky even when credit score is not extremely low.
- Some ambiguous observations are expected near the boundary.
- A deeper tree could fit more local details, but it may also overfit.

Confusion Matrix

	Predicted 0	Predicted 1
True 0	TN	FP
True 1	FN	TP

- Accuracy: overall correct classification rate.
- Precision: among predicted positives, how many are truly positive.
- Recall: among true positives, how many are found.
- F1: harmonic mean of precision and recall.

Student Example: Evaluation

		Predicted label	
		needs support	ready
Truelabel	needs support	11 TN	3 FP
	ready	1 FN	12 TP

Metric	Value
Accuracy	0.852
Precision	0.800
Recall	0.923
F1	0.857

- Test set size: 27.
- False negatives are important if missing support needs is costly.

Loan Case: Evaluation

		Predicted label	
		no default	default
Truelabel	no default	24 TN	4 FP
	default	9 FN	11 TP

Metric	Value
Accuracy	0.729
Precision	0.733
Recall	0.550
F1	0.629

- Recall is relatively low: some defaults are missed.
- In credit risk, false negatives may be more expensive than false positives.

How to Improve the Model

- Add more predictive features: income, employment length, loan amount, past delinquencies.
- Tune tree depth and minimum leaf size.
- Compare with logistic regression, random forests, and gradient boosting.
- Use cross-validation rather than one train/test split.
- Evaluate fairness and stability before operational use.

How to Run the Code

```
# Regenerate all datasets, metrics, and figures
```

```
/Users/shirongxu/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3 code/  
decision_tree_classification_demo.py
```

```
# Stop after a specific step
```

```
/Users/shirongxu/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/bin/python3 code/  
decision_tree_classification_demo.py --step 6
```

```
# Compile the slides
```

```
latexmk -xelatex -interaction=nonstopmode decision_tree_classification_slides.tex
```

Key Takeaways

- Classification predicts discrete labels from feature vectors.
- Decision trees use a sequence of interpretable feature tests.
- Training chooses splits that reduce impurity.
- Decision boundaries are axis-parallel for standard decision trees.
- Confusion matrices show what kinds of mistakes the model makes.

Discussion Questions

- ① When would accuracy be a misleading metric?
- ② In the loan case, which error is more costly: false positive or false negative?
- ③ What features would you add before using this model in a real application?
- ④ How deep should a tree be if the goal is explanation rather than prediction?